

EvoCracker

AI RESEARCH PROJECT

Phase 5 of 5 — The Game

PHASE 5

The Game

Playable web application, deployment infrastructure & evaluator demo guide

Deliverable Summary

- Completed end-to-end playable loop from calibration to evolved rounds.
- Integrated in-game analytics dashboard for algorithm and evolution inspection.
- Deployed to production on Vercel with Supabase backend and public GitHub repository.
- Assigned a default algorithm per enemy type — all 8 search algorithms are guaranteed visible in the Pathfinding dashboard from round one, regardless of genetic convergence.

Verification Checklist

- ✓ Core gameplay loop is complete and demonstrable.
- ✓ Dashboard requirement is satisfied in playable runtime.
- ✓ Project is live on Vercel and publicly accessible.
- ✓ Source code is available on GitHub.
- ✓ All 8 search algorithms are observable and comparable at all times via default enemy-algorithm mapping.

1. Live Deployment

EvoCracker is live and publicly accessible. No installation required — open the link in any modern browser.

Live App

evocracker.vercel.app

GitHub

github.com/AVM4D/EvoCracker

Stack

Vite + React + PixiJS frontend · Vercel hosting · Supabase backend · TypeScript throughout

2. Infrastructure

Vercel Deployment

The project is deployed as a static Vite application on Vercel with zero-config CI/CD from the GitHub repository.

Setting	Value
Build Command	npm run build
Output Directory	dist/
Framework Preset	Vite
Auto-Deploy	On push to main branch
Live URL	https://evocracker.vercel.app

Supabase Backend

Supabase provides the persistent data layer for player profiles, evolution history, and generation statistics. All data is linked to anonymous session identifiers — no account required to play.

Responsibility	Details
Player Profiles	Persists classified playstyle and raw telemetry across sessions.
Evolution History	Stores genome snapshots, fitness scores, and generation statistics per floor.
Session Linking	Anonymous session IDs — no login required.
Real-Time Sync	Dashboard analytics read live from Supabase tables.

GitHub Repository

The full source code is publicly available. The repository includes the complete Vite + React + PixiJS project, all AI modules, and the Genetic Algorithm implementation.

GitHub

[AVM4D/EvoCracker — EvoCracker: The Summoner's Trial](#)

3. Local Development

For local development and testing:

```
npm install
npm run dev
```

Local development URL:

```
http://127.0.0.1:5173/
```

Production build:

```
npm run build
```

Build output directory:

```
dist/
```

4. Core Gameplay Loop

The game loop matches the evaluator requirement end-to-end:

- 1 Calibration round starts. No enemies spawn. Telemetry recording begins immediately.
- 2 Player reaches the exit to finish calibration. `finalizeLearning()` triggers.
- 3 Telemetry is cleaned, preprocessed, and classified into a playstyle label.
- 4 Generation 1 enemies spawn using the calibrated profile and Playstyle-Adaptive Bias.
- 5 Player reaches the exit to finish the combat round. Defeating enemies is optional.
- 6 Enemy genomes evolve between rounds — fitness evaluated, crossover and mutation applied.

- 7

The dashboard visualizes generations, algorithm stats, and the player's profile in real time.

5. User Interface & Dashboard

The AI Analytics Dashboard is available in-game using the backtick (```) key.

Dashboard Tab	Contents
Algorithms	Algorithm reference, active enemy algorithms, runtime comparison.
Genomes	Population genes and fitness scores per agent.
Evolution	Fitness graph and generation-over-generation metrics.
Player	Classified playstyle, telemetry summary, exploration data.
Performance	FPS, enemy count, current iteration, difficulty index.

6. Sprint & Movement

The player can sprint with the Shift key.

Rule	Behaviour
While Held	Sprint stamina drains to 0.
Fully Depleted	Sprint is unavailable for 30 seconds.
After Recharge	Stamina returns to full — sprint is restored.
Calibration	Movement is boosted automatically so the setup round is faster.

7. Evaluator Demo Checklist

Recommended step-by-step flow for demonstrating EvoCracker to an evaluator:

- 1

Open the live app at evocracker.vercel.app — no installation required.
- 2

Complete calibration by reaching the exit. Observe that no enemies spawn during this phase.
- 3

Open the dashboard (``` key) → Player tab. Show that raw telemetry produced a classified playstyle label.

4

Play a combat round. Open Evolution tab — show the fitness chart and generation stats updating.

5

Open the Pathfinding tab — all 8 algorithms appear immediately because each enemy type has a hardcoded default. Compare Compute Time and Nodes Expanded across algorithms (e.g. A* vs. DFS vs. Hill Climbing).

6

Observe movement in gameplay — each default algorithm produces visibly different chase behaviour: DFS Stalkers dive deep, A* Hunters take optimal routes, Greedy BFS Flankers rush recklessly, Hill Climbing Ambushers creep locally.

7

Reach the exit on a combat floor. Show that the next generation spawns with updated genomes.

8

Point to the GitHub repository for full source review: github.com/AVM4D/EvoCracker